

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-1	UVa 10791	Minimum Sum LCM	唯一分解	陈锋
例 3-2	UVa 11582	Colossal Fibonacci Numbers	模算术；幂取模	刘汝佳
例 3-3	UVa 10375	Choose and Divide	模幂运算；素数判定	刘汝佳
例 3-4	UVa 1363	Joseph's Problem	数列求和	陈锋
例 3-5	UVa 1654	Pair of Integers	扩展欧几里得	陈锋
例 3-6	UVa 11440	Help Mr. Tomisu	素数筛法；阶乘的 phi 函数	刘汝佳
例 3-7	UVa 11426	GCD-Extreme(II)	gcd 函数；phi 函数	刘汝佳
例 3-8	UVa 10820	Send a Table	欧拉 phi 函数预处理	刘汝佳
例 3-9	UVa 10214	Trees in a Wood	欧拉 phi 函数	刘汝佳
例 3-10	UVa 11754	Code Feat	中国剩余定理	刘汝佳
例 3-11	NC 50316	A Horrible Poem	Hash；线性筛；素数分解	陈锋
例 3-12	SPOJ-VLATTICE	Visible Lattice Points	莫比乌斯反演	陈锋
例 3-13	HDU 4746	Mophues	莫比乌斯反演；GCD	陈锋
例 3-14	UVa 1457	Decrypt Messages	高次模方程；BSGS；原根	刘汝佳
例 3-15	SPOJ MOD	Power Modulo Inverted	扩展 BSGS；离散对数	陈锋

3.2 组合计数

组合计数是与算法关系最紧密的数学领域，许多经典问题的思路都与其紧密相关。本节给出一些经典例题及其实现。

例3-16 【编码解码问题】密码（Password, ACM/ICPC Daejon 2010, UVa 1262）

给出两个6行、5列的字母矩阵，找出满足如下条件的“密码”：密码中的每个字母在两个矩阵的对应列中均出现。例如，左数第2个字母必须在两个矩阵中的左数第2列中均出现。如图3-2所示，COMPU和DPMAG都满足条件。

A	Y	G	S	U
D	O	M	R	A
C	P	F	A	S
X	B	O	D	G
W	D	Y	P	K
P	R	X	W	O

C	B	O	P	T
D	O	S	B	G
G	T	R	A	R
A	P	M	M	S
W	S	X	N	U
E	F	G	H	I

图3-2 满足条件的密码

字典序最小的5个满足条件的密码分别是ABGAG、ABGAS、ABGAU、ABGPG和ABGPS。给定 k ($1 \leq k \leq 7777$)，你的任务是找出字典序第 k 小的密码。如果不存在，输出“NO”。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
using namespace std;

int k, cnt;
char p[2][6][9], ans[9];
const int SIGMA = 26;

bool dfs(int col) {
    if (col == 5) {
        if (++cnt == k) {
            ans[col] = '\0', puts(ans);
            return true; // 找到了
        }
        return false;
    }
    bool vis[2][SIGMA];
    memset(vis, 0, sizeof(vis));
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 6; j++) vis[i][p[i][j][col] - 'A'] = true;
    for (int i = 0; i < SIGMA; i++)
        if (vis[0][i] && vis[1][i]) {
            ans[col] = 'A' + i;
            if (dfs(col + 1)) return true;
        }
    return false;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &k);
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 6; j++) scanf("%s", p[i][j]);
        cnt = 0;
        if (!dfs(0)) puts("NO");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-8
同类题目：Code, POJ 1850
*/

```

例3-17 【支持四则运算的大整数类；平面的欧拉定理；计数】多少块土地 (How Many Pieces of Land, UVa 10213)

有一块椭圆形的地。在边界上选 n ($0 \leq n < 2^{31}$) 个点并两两连接, 得到 $n(n-1)/2$ 条线段。它们最多能把地分成多少个部分? 如图3-3所示, $n=6$ 时最多能分成31块。

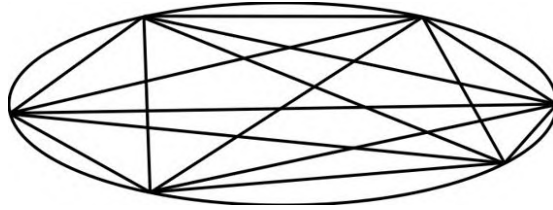


图3-3 $n=6$ 时可分的土地块数

【代码实现】

```
// 刘汝佳  
#include <bits/stdc++.h>
```

```

using namespace std;
typedef long long LL;
struct bigint {
    static const int base = 1e9, base_digits = 9;
    vector<int> a;
    int sign;
    bigint() : sign(1) {}
    bigint(LL v) { *this = v; }
    bigint(const string &s) { read(s); }
    void operator=(const bigint &v) {
        sign = v.sign;
        a = v.a;
    }

    void operator=(LL v) {
        sign = 1;
        if (v < 0) sign = -1, v = -v;
        for (; v > 0; v = v / base) a.push_back(v % base);
    }

    bigint operator+(const bigint &v) const {
        if (sign == v.sign) {
            bigint res = v;

            for (int i=0, carry=0; i < (int)max(a.size(),v.a.size())||carry; ++i){
                if (i == (int)res.a.size()) res.a.push_back(0);
                res.a[i] += carry + (i < (int)a.size() ? a[i] : 0);
                carry = res.a[i] >= base;
                if (carry) res.a[i] -= base;
            }
            return res;
        }
        return *this - (-v);
    }

    bigint operator-(const bigint &v) const {
        if (sign == v.sign) {
            if (abs() >= v.abs()) {
                bigint res = *this;
                for (int i = 0, carry = 0; i < (int)v.a.size() || carry; ++i) {
                    res.a[i] -= carry + (i < (int)v.a.size() ? v.a[i] : 0);
                    carry = res.a[i] < 0;
                    if (carry) res.a[i] += base;
                }
                res.trim();
                return res;
            }
            return -(v - *this);
        }
    }

```

```

    return *this + (-v);
}

void operator*=(int v) {
    if (v < 0) sign = -sign, v = -v;
    for (int i = 0, carry = 0; i < (int)a.size() || carry; ++i) {
        if (i == (int)a.size()) a.push_back(0);
        LL cur = a[i] * (LL)v + carry;
        carry = (int)(cur / base);
        a[i] = (int)(cur % base);
        // asm("divl %%ecx" : "=a"(carry), "=d"(a[i]) : "A"(cur), "c"(base))
    }
    trim();
}

bigint operator*(int v) const {
    bigint res = *this;
    res *= v;
    return res;
}

friend pair<bigint, bigint> divmod(const bigint &a1, const bigint &b1) {
    int norm = base / (b1.a.back() + 1);
    bigint a = a1.abs() * norm;
    bigint b = b1.abs() * norm;
    bigint q, r;
    q.a.resize(a.a.size());

    for (int i = a.a.size() - 1; i >= 0; i--) {
        r *= base;
        r += a.a[i];
        int s1 = r.a.size() <= b.a.size() ? 0 : r.a[b.a.size()];
        int s2 = r.a.size() <= b.a.size() - 1 ? 0 : r.a[b.a.size() - 1];
        int d = ((LL)base * s1 + s2) / b.a.back();
        r -= b * d;
        while (r < 0) r += b, --d;
        q.a[i] = d;
    }

    q.sign = a1.sign * b1.sign;
    r.sign = a1.sign;
    q.trim();
    r.trim();
    return make_pair(q, r / norm);
}

bigint operator/(const bigint &v) const { return divmod(*this, v).first; }

bigint operator%(const bigint &v) const { return divmod(*this, v).second; }

```

```

void operator/=(int v) {
    if (v < 0) sign = -sign, v = -v;
    for (int i = (int)a.size() - 1, rem = 0; i >= 0; --i) {
        LL cur = a[i] + rem * (LL)base;
        a[i] = (int)(cur / v);
        rem = (int)(cur % v);
    }
    trim();
}

bigint operator/(int v) const {
    bigint res = *this;
    res /= v;
    return res;
}

int operator%(int v) const {
    if (v < 0) v = -v;
    int m = 0;
    for (int i = a.size() - 1; i >= 0; --i) m = (a[i] + m * (LL)base) % v;
    return m * sign;
}

void operator+=(const bigint &v) { *this = *this + v; }
void operator-=(const bigint &v) { *this = *this - v; }
void operator*=(const bigint &v) { *this = *this * v; }
void operator/=(const bigint &v) { *this = *this / v; }

bool operator<(const bigint &v) const {
    if (sign != v.sign) return sign < v.sign;
    if (a.size() != v.a.size()) return a.size() * sign < v.a.size() * v.sign;
    for (int i = a.size() - 1; i >= 0; i--)
        if (a[i] != v.a[i]) return a[i] * sign < v.a[i] * sign;
    return false;
}

bool operator>(const bigint &v) const { return v < *this; }
bool operator<=(const bigint &v) const { return !(v < *this); }
bool operator>=(const bigint &v) const { return !(*this < v); }
bool operator==(const bigint &v) const {
    return !(*this < v) && !(v < *this);
}

bool operator!=(const bigint &v) const { return *this < v || v < *this; }

void trim() {
    while (!a.empty() && !a.back()) a.pop_back();
    if (a.empty()) sign = 1;
}

```

```

bool isZero() const { return a.empty() || (a.size() == 1 && !a[0]); }

bigint operator-() const {
    bigint res = *this;
    res.sign = -sign;
    return res;
}

bigint abs() const {
    bigint res = *this;
    res.sign *= res.sign;
    return res;
}

LL longValue() const {
    LL res = 0;
    for (int i = a.size() - 1; i >= 0; i--) res = res * base + a[i];
    return res * sign;
}

friend bigint gcd(const bigint &a, const bigint &b) {
    return b.isZero() ? a : gcd(b, a % b);
}

friend bigint lcm(const bigint &a, const bigint &b) {
    return a / gcd(a, b) * b;
}

void read(const string &s) {
    sign = 1;
    a.clear();
    int pos = 0;
    while (pos < (int)s.size() && (s[pos] == '-' || s[pos] == '+')) {
        if (s[pos] == '-') sign = -sign;
        ++pos;
    }
    for (int i = s.size() - 1; i >= pos; i -= base_digits) {
        int x = 0;
        for (int j = max(pos, i - base_digits + 1); j <= i; j++)
            x = x * 10 + s[j] - '0';
        a.push_back(x);
    }
    trim();
}

friend istream &operator>>(istream &stream, bigint &v) {
    string s;
    stream >> s;
    v.read(s);
}

```



```

    return stream;
}

friend ostream &operator<<(ostream &stream, const bigint &v) {
    if (v.sign == -1) stream << '-';
    stream << (v.a.empty() ? 0 : v.a.back());
    for (int i = (int)v.a.size() - 2; i >= 0; --i)
        stream << setw(base_digits) << setfill('0') << v.a[i];
    return stream;
}

static vector<int> convert_base(const vector<int> &a, int old_digits,
                                int new_digits) {
    vector<LL> p(max(old_digits, new_digits) + 1);
    p[0] = 1;
    for (int i = 1; i < (int)p.size(); i++) p[i] = p[i - 1] * 10;
    vector<int> res;
    LL cur = 0;
    int cur_digits = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        cur += a[i] * p[cur_digits];
        cur_digits += old_digits;
        while (cur_digits >= new_digits) {
            res.push_back(int(cur % p[new_digits]));
            cur /= p[new_digits];
            cur_digits -= new_digits;
        }
    }
    res.push_back((int)cur);
    while (!res.empty() && !res.back()) res.pop_back();
    return res;
}

typedef vector<LL> vll;

static vll karatsubaMultiply(const vll &a, const vll &b) {
    int n = a.size();
    vll res(n + n);
    if (n <= 32) {
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) res[i + j] += a[i] * b[j];
        return res;
    }

    int k = n >> 1;
    vll a1(a.begin(), a.begin() + k);
    vll a2(a.begin() + k, a.end());
    vll b1(b.begin(), b.begin() + k);
    vll b2(b.begin() + k, b.end());

```

```

    vll albl = karatsubaMultiply(a1, b1);
    vll a2b2 = karatsubaMultiply(a2, b2);

    for (int i = 0; i < k; i++) a2[i] += a1[i];
    for (int i = 0; i < k; i++) b2[i] += b1[i];

    vll r = karatsubaMultiply(a2, b2);
    for (int i = 0; i < (int)albl.size(); i++) r[i] -= albl[i];
    for (int i = 0; i < (int)a2b2.size(); i++) r[i] -= a2b2[i];

    for (int i = 0; i < (int)r.size(); i++) res[i + k] += r[i];
    for (int i = 0; i < (int)albl.size(); i++) res[i] += albl[i];
    for (int i = 0; i < (int)a2b2.size(); i++) res[i + n] += a2b2[i];
    return res;
}

bigint operator*(const bigint &v) const {
    vector<int> a6 = convert_base(this->a, base_digits, 6);
    vector<int> b6 = convert_base(v.a, base_digits, 6);
    vll a(a6.begin(), a6.end());
    vll b(b6.begin(), b6.end());
    while (a.size() < b.size()) a.push_back(0);
    while (b.size() < a.size()) b.push_back(0);
    while (a.size() & (a.size() - 1)) a.push_back(0), b.push_back(0);
    vll c = karatsubaMultiply(a, b);
    bigint res;
    res.sign = sign * v.sign;
    for (size_t i = 0, carry = 0; i < c.size(); i++) {
        LL cur = c[i] + carry;
        res.a.push_back((int)(cur % 10000000));
        carry = (int)(cur / 10000000);
    }
    res.a = convert_base(res.a, 6, base_digits);
    res.trim();
    return res;
}

};

// V-E+F=2, F=2-V+E-1=1+E-V
bigint solve(LL n) {
    bigint ans = n;
    ans *= (n - 1);
    ans *= bigint(n * n - 5 * n + 18);
    ans /= 24;
    ans += 1;
    return ans;
}

```

```

int main() {
    int s, n;
    cin >> s;
    while (cin >> n) cout << solve(n) << endl;
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-23
同类题目：Edge Case, ACM/ICPC NWERC 2012, UVa 1646
          That Nice Euler Circuit, 上海2004, POJ 2284
*/

```

例3-18 【有根树计数】统计有根树（Count，成都2012, UVa 1645)

输入 n ($n \leq 1000$)，统计有多少个 n 结点的有根树，使得每个深度中所有结点的子结点数相同。例如， $n=4$ 时有3棵，如图3-4所示； $n=7$ 时有10棵。输出数目除以 10^9+7 的余数。

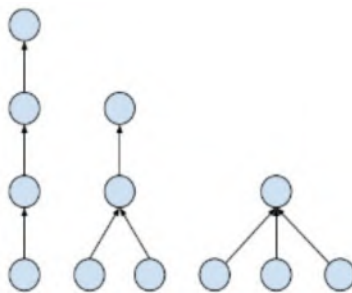


图3-4 $n=7$ 时有10棵树

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7, MAXN = 1024;
typedef long long LL;
LL A[MAXN];
void init() {
    A[1] = A[2] = 1;
    for (int i = 3; i < MAXN; ++i) {
        LL& a = A[i];
        a = 0;
        for (int j = 1; j < i; j++)
            if ((i - 1) % j == 0) {a += A[j]} %= MOD;
    }
}

int main() {
    init();
    for (int n, t = 1; scanf("%d", &n) == 1; t++)
        printf("Case %d: %lld\n", t, A[n]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 10-10
同类题目：Exploring Pyramids, ACM/ICPC NEERC 2005, Codeforces Gym 101334E
*/

```

例 3-19 【反向计算二项式系数】二项式系数（Binomial coefficients, ACM/ICPC NWERC 2011, UVa 1649）

输入 m ($2 \leq m \leq 10^{15}$)，求所有使得 $C(n, k) = m$ 的 (n, k) 。输出按照 n 升序排列，当 n 相同时按 k 升序排列。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
typedef pair<LL, LL> LPair;
typedef map<LL, set<LL>>::iterator MIT;

LL gcd(LL a, LL b) { return b == 0 ? a : gcd(b, a % b); }
int binomial_cmp(LL n, LL k, LL m) { // C(n, k) < M: -1, = 0: 0, > 0: 1
    LL C = 1;
    for (int i = 1; i <= k; i++) { // C *= (n-i+1)/i
        LL g = gcd(n - i + 1, i), b = (n - i + 1) / g;
        C /= (i / g);
        if (C > LONG_MAX / b) return 1; // 提前判断溢出
        C *= b;
        if (C > m) return 1;
    }
    if (C == m) return 0; // assert(C < m)
    return -1;
}

void solve(LL M, map<LL, set<LL>>& ans, int& sz) {
    ans.clear();
    sz = 0;
    if (M == 2) {
        ans[2].insert(1), sz = 1;
        return;
    }
    int k = 2;
    while (true) {
        LL L = 2 * k, R = M, n = 0;
        // if (is_debug()) printf("L:%lld, R:%lld, k:%d\n", L, R, k);
        if (binomial_cmp(L, k, M) > 0) break;
        while (L < R && !n) { // C(L, k) <= M < C(R, k)
            LL mid = L + (R - L) / 2;
            int cmp = binomial_cmp(mid, k, M);
            if (cmp == 0)
                n = mid;
            else if (cmp < 0)
                L = mid + 1;
            else if (cmp > 0)
                R = mid;
        }
        if (n) ans[n].insert(k), ans[n].insert(n - k);
        k++;
    }
}

```

```

    ans[M].insert(1), ans[M].insert(M - 1);
    for (MIT p = ans.begin(); p != ans.end(); p++) sz += p->second.size();
}

int main() {
    int T;
    scanf("%d\n", &T);
    map<LL, set<LL>> ans;
    for (LL M; scanf("%lld\n", &M) == 1;) {
        int sz = 0;
        solve(M, ans, sz);
        printf("%d\n", sz);
        int cnt = 0;
        for (MIT p = ans.begin(); p != ans.end(); p++) {
            const set<LL> s = p->second;
            for (set<LL>::iterator it = s.begin(); it != s.end(); it++) {
                if (cnt++) printf(" ");
                printf("%lld,%lld", p->first, *it);
            }
        }
        puts("");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 10-21
注意：本题的溢出判断技巧，以及如何用整数运算来避免浮点误差
同类题目：Irrelevant Elements, ACM/ICPC NEERC 2004, UVa 1635
*/

```

例3-20 【容斥原理】拉拉队 (Cheerleaders, UVa 11806)

在一个 m 行 n 列的矩形网格中放 k 个相同的石子，问有多少种放法。每个格子最多放一个石子，所有石子都要用完，并且第一行、最后一行、第一列、最后一列都得有石子。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const int MOD = 1000007, MAXC = 400 + 4;
int M, N, K;
LL C[MAXC][MAXC];
void init() {
    C[1][0] = C[1][1] = 1;
    for (int n = 2; n < MAXC; n++) {
        C[n][0] = 1;
        for (int k = 1; k <= n; k++)
            C[n][k] = (C[n - 1][k - 1] + C[n - 1][k]) % MOD;
    }
}
inline LL CK(int m, int n) { return C[m * n][K]; } // get C(m*n, k)

LL solve() {
    if (K < 2 || K > M * N) return 0;
    LL S = C[M * N][K];
    /*S -= (2*CK(M, N-1) + 2*CK(M-1, N))%MOD; // A,B,C,D
    S += (4*CK(M-1,N-1) + CK(M-2, N) + CK(M,N-2))%MOD; // AB,AC,AD,BC,BD,CD
    S -= (2*CK(M-2,N-1) + 2*CK(M-1, N-2))%MOD; // ABC, ABD, ACD, BCD
    S += CK(M-2, N-2); // ABCD
    for (int b = 1; b < 16; b++) { // 3210 LRTB
        int cnt = 0, m = M, n = N;
        if (b & 8) --m, ++cnt;
        if (b & 4) --m, ++cnt;
        if (b & 2) --n, ++cnt;
        if (b & 1) --n, ++cnt;
        LL x = C[m * n][K];
        if (cnt % 2) x = -x;
        S = (S + MOD + x) % MOD;
    }
    return S;
}

int main() {
    int T;

    init();
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> T;
    for (int t = 1; t <= T; t++) {
        cin >> M >> N >> K;
        printf("Case %d: %lld\n", t, solve());
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》2.1节例题3
同类题目: Deranged Exams, ACM/ICPC Greater NY 2013
*/

```

例3-21 【多项式；差分；整除性】总是整数 (Always an Integer, World Finals 2008, UVa 1069)

组合数学主要研究计数问题。比如，从 n 个人中选两个人，有多少种方法；圆周上有 n 个点，两两相连之后最多能把圆分成多少部分（见图3-5）；有一个金字塔，从塔顶开始每一层分别有 $1 \times 1, 2 \times 2, \dots, n \times n$ 个小立方体，问一共有多少个小立方体。

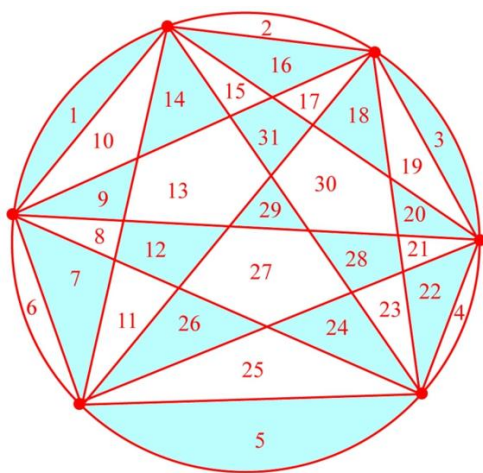


图3-5 用圆周上的 n 个点分割圆

很多问题的答案都可以写成 n 的简单多项式。比如，上述第1个问题的答案是 $n(n-1)/2$ ，也就是 $(n^2-n)/2$ ；第2个问题的答案是 $(n^4-6n^3+23n^2-18n+24)/24$ ；第3个问题的答案是 $n(n+1)(2n+1)/6$ ，即 $(2n^3+2n^2+n)/6$ 。

由于上述3个多项式是计数问题的答案，因此当 n 取任意正整数时，这些多项式的值都是整数。当然，对于其他多项式，这个性质并不一定成立。

给定一个形如 P/D （其中， P 是 n 的整系数多项式， D 是正整数）的多项式，判断它是否能在所有正整数处取到整数值。

【代码实现】


```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

struct Polynomial {
    vector<int> a, p;
    void parse_polynomial(string expr) {
        int i = 0, len = expr.size();
        while (i < len) {
            int sign = 1, v = 0;
            if (expr[i] == '+') i++;
            if (expr[i] == '-') sign = -1, i++;
            while (i < len && isdigit(expr[i])) v = v * 10 + expr[i++] - '0';
            // 系数的绝对值
            if (i == len) {
                a.push_back(v), p.push_back(0);
                continue;
            }
            assert(expr[i] == 'n');
            if (v == 0) v = 1;
            v *= sign;
            if (expr[i++] == '^') {
                a.push_back(v), v = 0;
            }
        }
    }
};

// 第 i 项为 a[i] * n^p[i]
// 解析多项式 (不带括号)
// 每次循环体解析一个 a * n^p
// 常数项
// 无系数, 按 1 处理
// 有指数项
// 清空 v, 接下来用 v 保存指数

```

```

        i++;
        while (i < len && isdigit(expr[i])) v = v * 10 + expr[i++] - '0';
        p.push_back(v);
    } else // 无指数项
        a.push_back(v), p.push_back(1);
    }
}

int mod(int x, int MOD) { // 计算 f(x) 除以 MOD 的余数
    int ans = 0;
    for (size_t i = 0; i < a.size(); i++) {
        int m = a[i];
        for (int j = 0; j < p[i]; j++) m = (LL)m * x % MOD;
        ans = ((LL)ans + m) % MOD; // 注意避免加法也可能会溢出
    }
    return ans;
}

bool check(string expr) {
    int p = expr.find('/');
    Polynomial poly;
    poly.parse_polynomial(expr.substr(1, p - 2));
    int D = atoi(expr.substr(p + 1).c_str());
    for (int i = 1; i <= poly.p[0] + 1; i++)
        if (poly.mod(i, D) != 0) return false;
    return true;
}

int main() {
    string expr;
    for (int kase = 1; cin >> expr; kase++) {
        if (expr[0] == '.') break;
        printf("Case %d: ", kase);
        if (check(expr))
            puts("Always an integer");
        else
            puts("Not always an integer");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.3.1 节例题 8
同类题目：Complete the sequence!, POJ 1398
*/

```

本节例题列表